PSAI_122

INK!/PALLET/SOLIDITY PERFORMANCE BENCHMARKING

- **Team Name:** Talentica Software
- **Payment Address:** 0x8bd54ec34A35f3A2f668A33d9578b5C3A6b730dE
- **Level:** 1

PROJECT OVERVIEW :

Proposal for the RFP titled ink!/pallet/solidity performance benchmarking.

OVERVIEW

There are multiple ways to implement the logic in substrate i.e using pallets or ink smart contracts, or even writing solidity code and compiling it to WASM with the help of a solang compiler. We have to benchmark the performance metrics of the logic implemented using each of the above methods. This will help new developers to decide the best tool to implement the logic.

PROJECT DETAILS

We will use following tools to benchmark the ink smart contract/pallet: - Ink Smart Contract - Smart Bench - Pallet - Substrate Runtime Benchmarking Framework

TEAM MEMBERS

- Debasish Ray Chawdhuri - Blockchain and Cryptography Researcher
- Pankaj Mendki -Blockchain expert
- Nikhil Desai - Blockchain Developer https://github.com/Nikhil-Desai-Talentica
- Amit Singh - Blockchain Developer https://github.com/iamit-singh

CONTACT

- **Contact Name:** Nikhil Desai
- **Contact Email:** Nikhil.Desai@talentica.com
- **Website:** https://www.talentica.com/

LEGAL STRUCTURE

- **Registered Address:** B-7/8, Anmol Pride, Baner Road, Baner, Pune, Maharashtra 411045, India
- **Registered Legal Entity:** Talentica Software India Pvt. Ltd.

TEAM CODE REPOS

- provenance-usecase
- simpleblockchain

TEAM LINKEDIN PROFILES (IF AVAILABLE)

- Debasish Ray Chawdhuri - https://www.linkedin.com/in/debasish-ray-chawdhuri-4b444316/
- Pankaj Mendki - https://www.linkedin.com/in/pankaj-mendki-8576363/
- Nikhil Desai - https://www.linkedin.com/in/nikhil-desai-1209a38b/
- Amit Singh - https://www.linkedin.com/in/amit-singh-3637a898/

DEVELOPMENT STATUS :

We have explored the tools specified in the RFP. We have also come up with a high-level implementation plan and will start implementing it soon.

DEVELOPMENT ROADMAP :

Number	Objective	Deliverable	Time Estimate
1.	Create a unified framework to work with both, Substrate Runtime Benchmarking Framework and Smart-bench	Milestone 1, Deliverable 4	3 days
2.	Test it with existing pallets	Milestone 1, Deliverable 4	2 days
3.	Create new pallets for benchmarking	Milestone 1, Deliverable 2	3 days
4.	Test it with the new pallets	Milestone 1, Deliverable 2	2 days
5.	Create new Ink smart contracts for benchmarking	Milestone 1, Deliverable 3	3 days
6.	Test it with the new Ink smart contracts	Milestone 1, Deliverable 3	2 days
	Adapt Smart Bench to work with		

7.	arbitrary Solidity smart contracts	Milestone 1, Deliverable 4	1 week
8.	Create new Solidity smart contracts for benchmarking	Milestone 2, Deliverable 1	3 days
9.	Test it with the new Solidity smart contracts	Milestone 2, Deliverable 1	2 days
10.	Adapt the framework to work with the above smart contracts	Milestone 2, Deliverable 2	1 Week

OVERVIEW

- **Total Estimated Duration:** 6 weeks
- **Full-Time Equivalent (FTE):** 1
- **Total Costs:** 10,000 DAI

Languages

- Rust
- Solidity

MILESTONES

MILESTONE 1 — BASIC BENCHMARKING

- **Estimated duration:** 3 weeks
- **FTE:** 1
- **Costs:** 5000 DAI

Number	Deliverable	Specification
0a.	**License**	Apache 2.0
0b.	**Documentation**	We will provide both **inline documentation** of the code and a **live demo**.
0c.	**Testing Guide**	Core functions will be fully covered by unit tests to ensure functionality and robustness. In the guide, we will describe how to run these tests.
0e.	**Article**	We will publish an **article** that explains how each implementation technique performs, and when to choose which.
1.	**Performance metrics**	After examining the existing benchmarking tools and discussing with their developers, we believe weight alone serves as a good metric to analyse performance.
2.	**Pallet: Basic Read & Write**	We will create some pallets which expose simple methods for writing to storage and reading from on-chain storage.
3.	**Ink Smart Contract: Basic Read & Write**	We will create some ink smart contracts which expose simple methods for writing to storage and reading from on-chain storage.
4.	Library: Benchmarking	We will deliver a Rust library that allows calling both the pallet's extrinsics and contract's public methods multiple times and return the benchmarks.

MILESTONE 2 — INTEGRATE NATIVE SOLIDITY & SOLANG-COMPILED SOLIDITY

- **Estimated duration:** 3 weeks
- **FTE:** 1
- **Costs:** 5000 DAI

Number	Deliverable	Specification
0a.	**License**	Apache 2.0
0b.	**Documentation**	We will provide both **inline documentation** of the code and a **live demo**.
0c.	**Testing Guide**	Core functions will be fully covered by unit tests to ensure functionality and robustness. In the guide, we will describe how to run these tests.
0e.	**Article**	We will publish an **article** that explains how each implementation technique performs, and when to choose which.
1.	**Solidity(native and WASM) Smart Contract: Basic Read & Write**	We will create some solidity smart contracts which expose simple methods for writing to storage and reading from on-chain storage.
2.	**Adapt the benchmarking framework**	We will adapt the benchmarking framework so it can handle WASM and native solidity code benchmarking.

ADDITIONAL INFORMATION:

Gautam Dhameja told us about the Grants program and encouraged us to apply to the same.